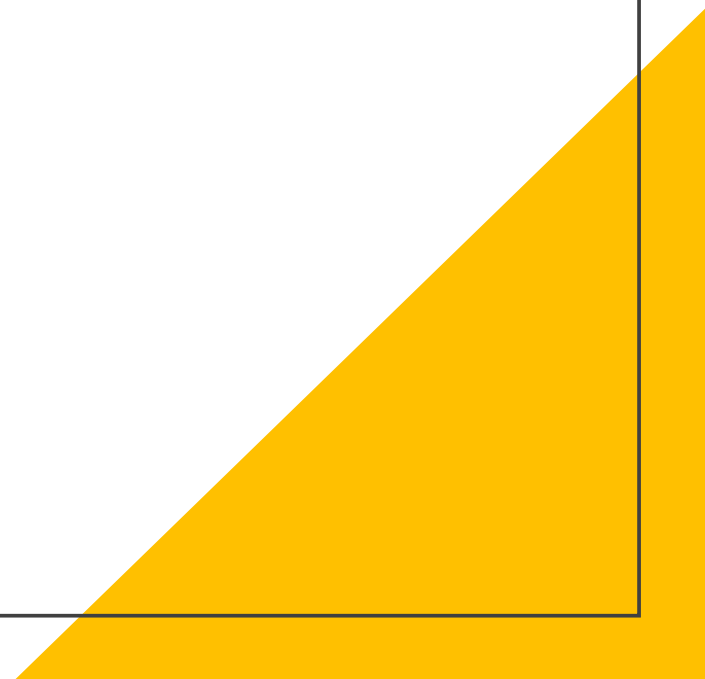


資料傳遞



路由資料傳遞

當你切換路由時有時候會需要將第一個頁面的資料內容傳遞給你二個頁面使用，但是又沒辦法在第二個頁面呼叫第一個頁面的資料。

這時候我們就需要去新增一個service用來擺放我的需要傳遞的資料，而這個service是每個頁面都可以去呼叫並且抓取裡面的內容，並且在這個service中不只能夠放變數也能夠擺放共用的方法。

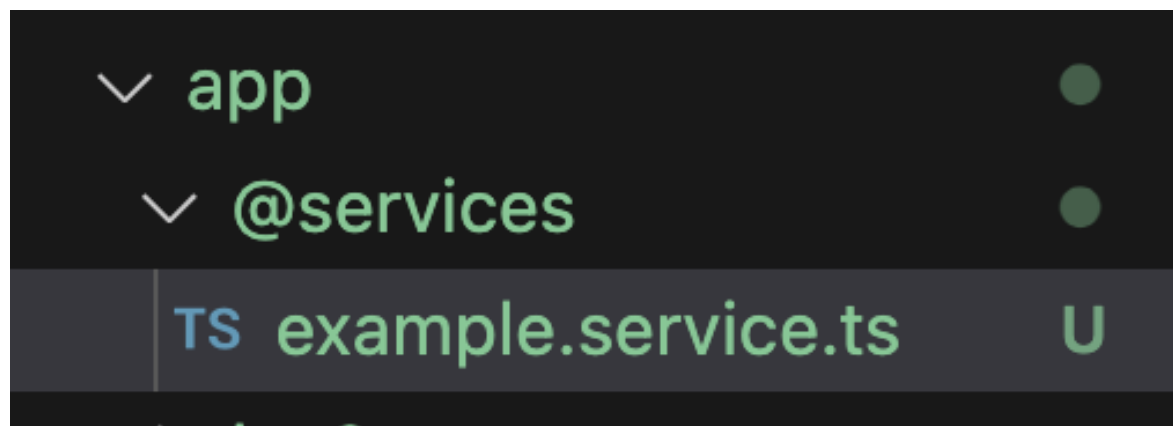
建立service

這邊建議可以新增一個資料夾擺放你建立的service，因為有可能你的專案不只有一個service檔案(正常來說會將功能切分開來)，所以我們先在專案的src目錄底下建立@services資料夾並且新增一個example.service.ts。

指令是 `ng g s 檔案路徑/檔案名稱`

檔案名稱不需要加service.ts系統會幫你加

以範例為例 `ng g s @services/example`



建立service

example-service.ts建立完之後他的內容預設請輸入以下內容，class後面是這個TS的名稱可自行取名(建議要跟你的檔案名稱一樣)。

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class ExampleService {

}
```

建立service

這時候你可以將你要傳遞的值在service裡面宣告一個一樣名稱的變數，當然名稱也可以不一樣不過一樣會比較好辨識(例如A頁面(first.component.html)要傳給B頁面(second.component.html)的東西是userName，那最好在service命名就也命名為userName)。

```
export class ExampleService {  
  userName!: string;  
}
```

example.service.ts

```
export class FirstComponent {  
  title = 'demo8';  
  userName: string = 'Allen';  
  constructor() {}  
}
```

first.component.ts

頁面傳遞資料

這時候請在你要傳遞資料的第一個頁面中注入你的service的檔案，並且將你要傳遞的資料塞進去service的變數內(如下圖)，這時候沒意外service裡面的userName就會等於Allen。

```
export class FirstComponent {  
  title = 'demo8';  
  userName: string = 'Allen';  
  constructor(private exampleService: ExampleService){}  
  
  ngOnInit(): void {  
    this.exampleService.userName = this.userName;  
  }  
}
```

first.component.ts

頁面傳遞資料

接著請到你要撈取的頁面中的TS一樣注入你的service的檔案，這時候就可以將service中的變數資料取出來使用了，不過需要小心service內的檔案是每個資料夾中都可以使用，所以需要確認這個資料的即時跟準確性。

```
export class SecondComponent{  
  constructor(private exampleService: ExampleService) {}  
  
  ngOnInit(): void {  
    console.log(this.exampleService.userName);  
  }  
}
```

second.component.ts

組件傳遞資料

剛剛是教各位路由的傳遞資料，現在要教各位當你在頁面使用組件時要怎麼將資料傳遞進組件與將資料從組件中傳遞出來。

這時候你就會使用到@Input將資料傳遞進去組件中還有@output將資料傳遞出來，這時候你會問為什麼組件中的內容不可以用service去抓取資料呢？

因為service抓資料只會抓一次沒辦法做到當service內容更新時，即時更新畫面所以組件才需要用到@Input與@output來去傳遞資料。

@Input

@Input就是將資料傳遞進去組件中，這個是會即時更新的。

首先我們需要現在組件中設定一個變數(這邊設定為value)，並且在前面加上@Input，還有導入Input的元件。

```
import { Component, Input } from '@angular/core';  
import { ExampleService } from '../@services/example.service';
```

```
export class SecondComponent {  
  @Input() value!: string;  
}
```

second.component.ts

@Input

接著在你使用的元件標籤上加上你剛剛宣告的@Input變數名稱並且使用[]將其包起來，裡面的內容就是綁定你使用此元件頁面的變數，這時候你就會發現當你頁面的變數更改時這個元件的變數也會跟著更改。

```
<app-second [value]="userName"></app-second>
```

first.component.html

練習1

根據右圖做出@Input的練習
頁面A的三個輸入欄位輸入的同時
頁面B組件中的內容也會同步更新顯示

頁面A

使用者名稱：
使用者Email：
使用者地址：

頁面B(組件)

使用者名稱：

使用者Email：

使用者地址：

@output

當你組件做了某件事情之後想要回傳組件想回傳的內容或者執行頁面的動作時，你就會需要使用到@output來觸發頁面的方法，一樣我們先來到組件的TS中，宣告一個變數名稱(這邊宣告為changeUserName)並且讓這個變數名稱 = output<類型>()。

```
import { Component, Input, output } from '@angular/core';
```

```
changeUserName = output<string>();
```

second.component.ts

@output

緊接著在你要觸發回傳更新內容或者觸發方法的地方使用這個變數別且.emit(回傳的內容)，因為我們剛剛寫的類型是string所以我裡面就直接放文字，這樣子就完成組件的觸發動作，接著就回到使用組件的頁面。

```
this.changeUserName.emit('BBAACC')
```

second.component.t

@output

我們看到使用組件的頁面，在該頁面的組件標籤中使用()裡面放組件的output變數名稱(我們宣告為changeUserName)，右邊就放我們觸發output後要觸發的方法(裡面的\$event代表的是剛剛組件方法裡面的內容)。

```
<app-second (changeUserName)="showNewName($event)" ></app-second>
```

first.component.html

```
showNewName(event: string) {  
  console.log(event);  
}
```

first.component.ts

練習2

根據右圖做出@output的練習
跟練習1相反 頁面B組件中的內容按了
按鈕之後要輸出到頁面A做呈現

頁面A

使用者名稱：

使用者Email：

使用者地址：

頁面B(組件)

使用者名稱：

使用者Email：

使用者地址：

output輸出

结束

